PROJET
UE : Vision par Ordinateur

Mars 2026

OULARE Djafarou (M2 Statistiques et Probabilités des nouvelles données)
BOHOU Christ (M2 Statistiques et Probabilités des nouvelles données)

Introduction

Ce projet consiste à développer une version interactive du jeu **Space Invaders**, contrôlée par des gestes de la main détectés via la vision par ordinateur. Pour ce faire :

- Dans un premier temps, des modèles de classification **MLP** (Perceptron Multicouche) et **CNN** (Réseaux de Neurones Convolutionnels) ont été implémentés à partir d'une base de données contenant des gestes de la main (droite, gauche, haut et bas). Ces modèles permettent de détecter avec précision les gestes dans les images, en classifiant chaque geste parmi les quatre catégories définies.
- Dans un second temps, ces modèles de classification seront intégrés dans un système de contrôle en temps réel du jeu, via des technologies telles que **WebSocket**, pour permettre à l'utilisateur d'interagir avec **Space Invaders** en utilisant des gestes.

1 Modélisation

1.1 Présentation du jeu de données

Le jeu de données utilisé, **Hand Gesture Landmark** (voir datasets), comprend une variété de gestes de la main classés en différentes catégories. Pour ce projet, nous avons restreint ces catégories aux gestes **Gauche**, **Droite**, **Haut**, et **Bas**. Chaque geste est constitué de plusieurs images qui ont été utilisées pour entraîner les modèles de classification. La base de données contient une base d'entraînement (2000 images), une base de test (300 images) et une base de validation (100 images). Dans chaque base de données, chaque image est de dimension 180x180x3 (où la hauteur est de 180 pixels, la largeur est de 180 pixels et les canaux de couleur sont RGB=3). Voir à la figure 1 quelques gestes de la main présents dans les bases de données :

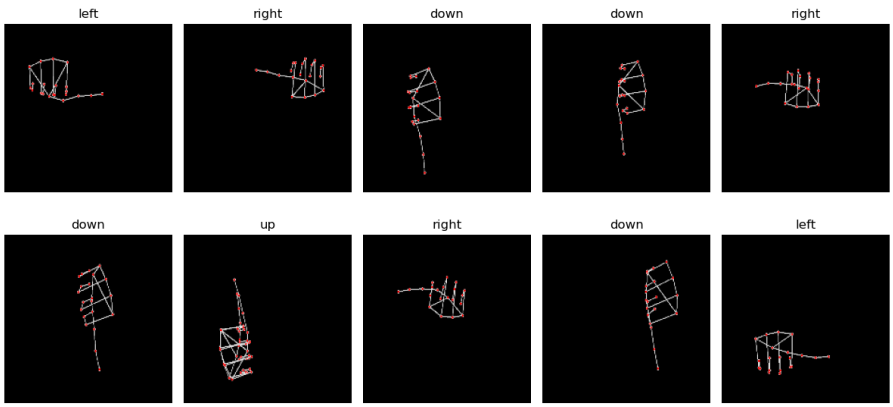

Figure 1: Quelques gestes de la main

1.2 Prétraitement des données

Avant l'entraînement des modèles, il s'avère capital de réaliser des prétraitements.

- La figure (2) montre la distribution de chaque classe dans chacune des trois bases. Nous remarquons que dans chacune des trois bases de données, les quatre classes sont uniformément distribuées (chaque classe contient le même nombre d'images). Ainsi, notre jeu de données ne souffre pas de déséquilibre de classes, ce qui pourrait conduire à un overfitting.

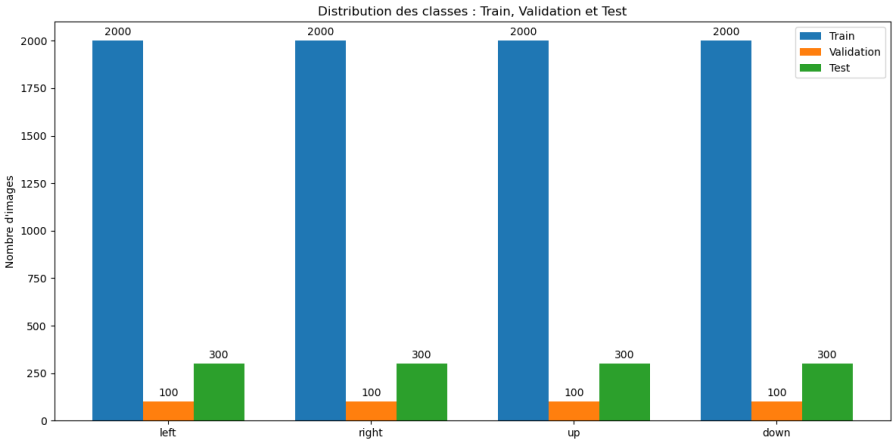

Figure 2: Distribution des classes : Train, Validation et Test

- Les images ont été normalisées afin qu'elles soient sur la même échelle. Cela permet de garantir que les valeurs des pixels sont dans une plage uniforme, ce qui aide à améliorer la convergence du modèle lors de l'entraînement.
- Comme les étiquettes des classes sont des catégories (left, right, up, down), elles ont été converties en vecteurs one-hot. Cette transformation permet de représenter les classes de manière binaire, ce qui est plus adapté aux modèles de classification.

1.3 Modèles

1.3.1 MLP (Multilayer Perceptron)

Le modèle **MLP (Multilayer Perceptron)** de ce projet est conçu pour classer les gestes de la main dans 4 catégories : gauche, droite, haut et bas. Il se compose de trois couches principales :

- **Couche de Flatten** : Aplatie les images de taille (180, 180, 3) en un vecteur 1D.
- **Couche cachée** : Une couche dense avec 128 neurones et la fonction d'activation ReLU.
- **Couche de sortie** : Une couche dense de 4 neurones avec la fonction Softmax pour la classification multi-classes.

L'optimisation est réalisée à l'aide de l'optimiseur Adam, avec une fonction de perte `categorical_crossentropy` et la précision (`accuracy`) comme métrique.

La figure 3 ci-dessous montre les courbes d'apprentissage du modèle MLP. La courbe de **perte** indique que la perte d'entraînement diminue rapidement, mais la perte de validation reste stable ou augmente légèrement, ce qui suggère un **surapprentissage**. La courbe de **précision** montre que la précision d'entraînement atteint rapidement un niveau élevé, tandis que la précision de validation stagne, confirmant également le **surapprentissage**. La figure 4 nous montre que le modèle MLP ne fait pas de bonnes prédictions sur la base de validation. Sur un échantillon de 6 images, il ne parvient à bien prédire que 2 d'entre elles. Cela confirme le surapprentissage observé précédemment sur la base d'entraînement.

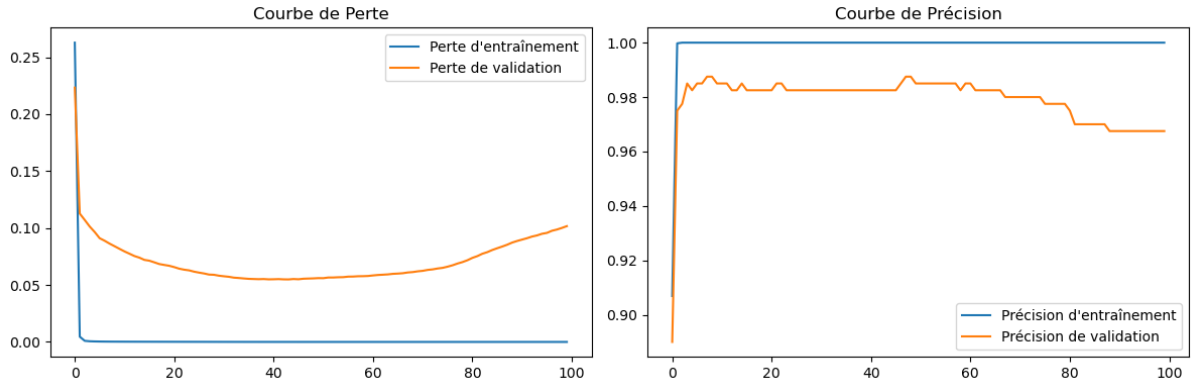

Figure 3: Courbes d'apprentissage du modèle MLP : Perte et Précision d'entraînement et de validation.

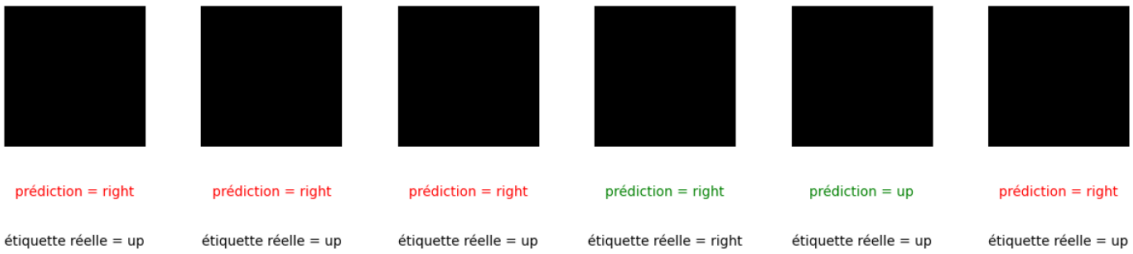

Figure 4: Prédictions du modèle MLP sur un échantillon de la base de validation

1.3.2 Modèle CNN (Convolutional Neural Network)

Le modèle **CNN** utilisé pour la classification des gestes de la main se compose de :

- **Couche Conv2D** : 32 filtres (3x3) avec activation `ReLU` pour l'extraction des caractéristiques.
- **MaxPooling2D** : Réduction de la taille des images (2x2).
- **Deuxième couche Conv2D** : 64 filtres (3x3) avec activation `ReLU`.
- **MaxPooling2D** : Nouvelle réduction de la taille.
- **Flatten** : Conversion en vecteur 1D pour les couches denses.
- **Couche Dense** : 128 neurones avec activation `ReLU`.
- **Couche de sortie** : 4 neurones avec activation `Softmax` pour la classification des 4 gestes.

L'optimisation est réalisée avec `Adam`, la fonction de perte est `categorical_crossentropy`, et la métrique est `accuracy`.

La figure 5 ci-dessous montre les courbes d'apprentissage du modèle CNN. Nous remarquons que la **perte d'entraînement** et la **perte de validation** diminuent de manière stable, ce qui indique que le modèle apprend efficacement tant sur les données d'entraînement que sur les données de validation. De plus, la **précision d'entraînement** ainsi que la **précision de validation** sont très proches l'une de l'autre, ce qui suggère que le modèle est capable de bien généraliser sur les données qu'il n'a pas vues pendant l'entraînement. Cela montre que le modèle CNN fonctionne bien à la fois sur la base d'entraînement et sur la base de validation.

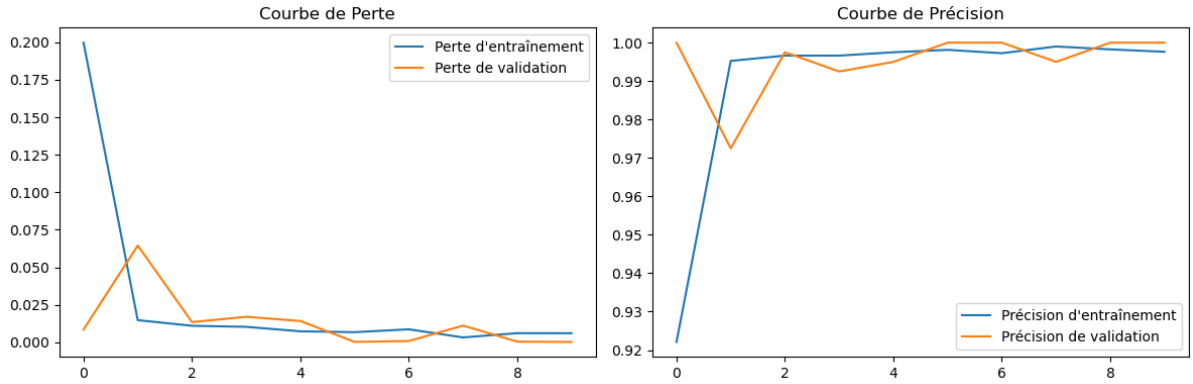

Figure 5: Courbes d'apprentissage du modèle CNN : Perte et Précision d'entraînement et de validation.

La figure 6 nous montre que le modèle CNN fait de bonnes prédictions sur la base de validation. Sur un échantillon de 6 images, il ne parvient à bien prédire les 6 images.

Figure 6: Prédictions du modèle CNN sur un échantillon de la base de validation

1.3.3 Récapitulatif : comparaisons et choix du modèle

Le tableau (1) présente un récapitulatif des performances des deux modèles. On observe que le modèle CNN présente une erreur plus faible et une meilleure précision, ce qui en fait le choix optimal pour l'intégration du jeu de détection de la position du doigt via Mediapipe et OpenCV. Nous allons donc utiliser le modèle CNN pour cette tâche.

Table 1: Comparaison des Performances sur l'Échantillon Test

Modèles	**Perte**	**Précision**
MLP (Multilayer Perceptron)	1.2058	80.75%
CNN (Convolutional Neural Network)	0.0870	97.25%

2 Intégration

Dans cette méthode d'intégration, la classification des gestes est effectuée en temps réel à l'aide d'un réseau de neurones convolutionnel (CNN). Voici les étapes clés de ce processus :

- **Capture des gestes en temps réel :** Les gestes de l'utilisateur sont capturés via une caméra et traités en temps réel à l'aide de **Mediapipe** et **OpenCV**. Mediapipe permet de détecter et de suivre la position des mains et des doigts, tandis qu'OpenCV gère l'affichage et le traitement des images.
- **Prétraitement des images :** Une région d'intérêt (ROI) est extraite de l'image capturée, redimensionnée et normalisée avant d'être envoyée au modèle CNN pour la classification.
- **Classification des gestes :** Le modèle CNN, préalablement entraîné, prédit le geste effectué en analysant la région d'intérêt. Le modèle retourne une probabilité pour chaque classe de geste (gauche, droite, haut, bas).

- **Affichage des résultats :** Les résultats de la classification sont affichés en temps réel sur l'écran. Le geste détecté, accompagné de sa probabilité, est superposé à l'image. Si la confiance dans la prédiction est élevée (au-dessus d'un seuil), le geste est affiché avec une barre de confiance.
- **Transmission des commandes :** Une fois le geste identifié, la commande correspondante est envoyée en temps réel via **WebSocket** au serveur du jeu, permettant de contrôler l'action dans le jeu en fonction des gestes détectés.

Cependant, cette méthode n'a pas donné des prédictions suffisamment fiables. La principale limitation réside dans le fait que nous n'avons pas généré nous-mêmes les images d'entraînement. La qualité des prédictions pourrait être améliorée en générant des images plus adaptées à l'environnement d'exécution réel.

Pour cela, nous avons proposé deux autres méthodes, qui sont documentées sur GitHub, à savoir :

- Une deuxième méthode dans laquelle nous avons généré nos propres images, effectué des CNN, et utilisé **Mediapipe** et **OpenCV** pour capturer les gestes en temps réel et transmettre les commandes au serveur du jeu ;
- Une troisième méthode qui repose sur un modèle pré-entraîné.

Conclusion

Dans le cadre de ce projet, nous avons :

- Implémenté des modèles de classification des gestes de la main à l'aide de **MLP** et **CNN**.
- Validé les performances des CNN pour détecter avec précision les gestes (**Gauche**, **Droite**, **Haut**, **Bas**).
- Préparé l'intégration de ce modèle dans un système interactif pour contrôler le jeu **Space Invaders** en temps réel à l'aide de gestes de la main.
- Entraîné deux autres méthodes, documentées sur GitHub.

Merci infiniment !